QALight

Frontend-розробка

Заняття 18. Класи і наслідування

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Класи і наслідування
Визначення класів. Методи класу. Наслідування класу. Статичні властивості та методи. Приватні та захищені властивості та методи. Статичні властивості. Робота з помилками.
Створення власного класу.

Класи і наслідування

Визначення класів. Методи класу. Наслідування класу. Статичні властивості та методи. Приватні та захищені властивості та методи. Статичні властивості.

Класи є шаблоном для створення об'єктів

Теорія: class JavaScript

Класи є шаблоном для створення об'єктів. Вони інкапсулюють дані за допомогою коду для роботи з цими даними. Класи в JS побудовані на **прототипах**, але також мають певний синтаксис і семантику, які є унікальними для класів.

Класи насправді є «спеціальними функціями», і так само, як ви можете визначити функціональні вирази та оголошення функцій, клас можна визначити двома способами: вираз класу або оголошення класу.

```
// Declaration
class Rectangle {
   constructor(height, width) {
     this.height = height;
     this.width = width;
   }
 }
```

Базовий синтаксис класу

Теорія: ООР

В об’єктно-орієнтованому програмуванні, клас – це спеціальна конструкція, яка використовується для групування пов’язаних змінних та функцій. При цьому, згідно з термінологією ООП, глобальні змінні класу (члени-змінні) називаються **полями** даних (також **властивостями** або **атрибутами**), а члени-функції називають **методами** класу.
В сучасному JavaScript клас визначається так само, як у більшості об’єктно-орієнтованих мов програмування. Основний синтаксис:

```
class MyClass {
        // методи класу
        constructor() { ... }
        method1() { ... }
        method2() { ... }
        method3() { ... }
 ...
    }
```

Щоб створити новий об’єкт з усіма перерахованими методами, використовуйте new MyClass().

Метод constructor()

Теорія: ООР

Метод **constructor**() викликається автоматично за допомогою **new**, в ньому ми можемо ініціалізувати об’єкт. Наприклад:

```
class User {
 constructor(name) {
   this.name = name;
 }

 sayHi() {
   console.log(this.name);
 }
}
// Використання:
// Створюється новий об’єкт.
let user = new User("Іван");
user.sayHi();
```

Чим насправді є class?

Теорія: ООР

У JavaScript клас є своєрідною функцією:

```
// User - це функція
console.log(typeof User); // function
```

Конструкція class User {...} в дійсності робить наступне:

- Створює функцію User. Код цієї функції береться з методу **constructor** (вважається порожнім, якщо ми не написали такий метод).
- Записує методи класу до **User.prototype**.
- Під час виклику методу, він береться з прототипу. Тобто, об’єкт має доступ до методів класу.

```
// метод конструктора
console.log(User === User.prototype.constructor); // true

// Методи знаходяться в User.prototype:
console.log(User.prototype.sayHi); // код sayHi методу

// у прототипі існує рівно два методи
console.log(Object.getOwnPropertyNames(User.prototype)); // constructor, sayHi
```

Тіло класу

Теорія: class JavaScript

Тіло класу – це частина, яка знаходиться у фігурних дужках {}. Тут ви визначаєте члени класу, такі як методи або конструктор.

Тіло класу виконується в **строгому режимі** навіть без директиви **use strict.**

Елемент класу можна охарактеризувати трьома аспектами:

Вид: Одержувач, установник, метод або поле
Розташування: статичне або екземпляр
Видимість: загальнодоступна або приватна

Разом вони додають до 16 можливих комбінацій.

Чисті функції

Практика: JavaScript функції

Чисті функції мають бути передбачуваними, тобто функція повинна завжди повертати те ж саме значення з однаковими аргументами. Спочатку розглянемо функцію, яка не є чистою:

```
let greeting = 'Hello';
function greet(name) {
 return greeting + ' ' + name;
}
console.log(greet('Paul')); // Hello Paul

Варіант чистої функції
function greet(greeting, name) {
 return greeting + ' ' + name;
}
console.log(greet('Howdy', 'Paul')); // Howdy Paul
```

Коли функція змінює щось поза межами функції, виникає **побічний ефект**. Чиста функція має бути передбачуваною та без побічних ефектів. Якщо будь-який із цих критеріїв не виконується, ми маємо справу з нечистою функцією.

Перепишемо клас User в чистих функціях

Теорія: ООР

Існують причини, чому class можна вважати синтаксичним цукром. Проте, існують відмінності.

- функція, що створена за допомогою class, позначена спеціальною внутрішньою властивістю **[[IsClassConstructor]]: true**.
- Рушій перевіряє цю властивість у різних місцях. Наприклад, на відміну від звичайної функції, її треба викликати з **new**:

```
// 1. Створимо функцію-конструктор
function User(name) {
 this.name = name;
}
// прототип функції має властивість "constructor" за замовчуванням, тому не потрібно її створювати
// 2. Додамо метод до прототипу
User.prototype.sayHi = function() {
 console.log(this.name);
};
console.log(typeof User); // function
User(); // Помилка: Конструктор класу User неможливо викликати без 'new'
console.log(User); // class User { ... }
```

Методи класу

Теорія: ООР

Методи класу **неперелічувані**.

Оголошення класу встановлює прапор **enumerable** у **false** для всіх методів в "**prototype**". Це добре тому, що коли ми перебираємо об’єкт за допомогою for..in, ми зазвичай не хочемо обробляти методи класу.

```
for (let item in user) {
  console.log(item);
}
```

Клас завжди **use strict**.

Весь код всередині конструкції класу автоматично знаходиться в суворому режимі.

Class Expression

Теорія: ООР

Так само, як функції, класи можуть бути визначені всередині іншого виразу, передані, повернуті, присвоєні тощо.

```
// Приклад class expression:

let User3 = class {
 sayHi() {
   console.log("Привіт");
 }
};
// Якщо Class Expression має назву, то її видно лише всередині класу:
let User4 = class MyClass {
 sayHi() {
   console.log(MyClass); // назву MyClass видно тільки всередині класу
 }
};
new User4().sayHi(); // працює, показує визначення MyClass
console.log(MyClass); // помилка, назву MyClass не видно за межами класу
```

Динамічні класи

Теорія: ООР

```
// Ми можемо навіть зробити класи динамічними “на вимогу”, наприклад:

function makeClass(phrase) {
 // оголошуємо клас і повертаємо його
 return class {
   sayHi() {
     console.log(phrase);
   }
 };
}

// Створюємо новий клас
let User = makeClass("Привіт");

new User().sayHi(); // Привіт
```

Геттери

Теорія: class JavaScript

Подібно до літералів об’єктів, класи можуть включати геттери/сеттери, обчислені атрибути тощо.

Синтаксис **get** пов'язує властивість об'єкта з функцією, яка буде викликана, коли ця властивість шукається.

{ get prop() { /* … */ } }

{ get [expression]() { /* … */ } }

Геттер повинен мати рівно нуль параметрів.

```
class User {

 constructor(name) {
    this.name = name;  // викликає сеттер
 }
 get name() {
   return this._name;
 }
```

Сеттери

Теорія: ООР

Синтаксис **set** прив’язує властивість об’єкта до функції, яку потрібно викликати, коли є спроба встановити цю властивість.
{ set prop(val) { /* … */ } } | { set [expression](val) { /* … */ } }
Сетер повинен мати рівно один параметр.

```
    set(key,value) {
        return localStorage.setItem(key,JSON.stringify(value));
   }
 set name(value) {
   if (value.length < 4) {
     console.log("Ім’я занадто коротке.");
     return;
   }
   this._name = value;
 }
}
let user = new User("Іван");
console.log(user.name); // Іван
user = new User(""); // Ім’я занадто коротке.
```

Обчислені назви […]

Теорія: ООР

Приклад з **обчисленою назвою методу** за допомогою використання дужок [...]:

```
class User {
 ['say' + 'Hi']() {
   console.log("Привіт");
 }
}

new User().sayHi();
```

Поля класу

Теорія: ООР

Поля класу – це синтаксис, який дозволяє додавати будь-які властивості:

```
class User {
  // просто пишемо <property name> = <value> в оголошенні
  name = "Іван";

  sayHi() {
    console.log(`Привіт, ${this.name}!`);
  }
}

new User().sayHi(); // Привіт, Іван!

// поля класу задаються в окремих об’єктах, а не в User.prototype:
let user = new User();
console.log(user.name); // Іван
console.log(User.prototype.name); // undefined
```

Наслідування класу

Теорія: ООР

Наслідування класу – це коли один клас розширює інший, тобто, ми можемо створити нову функціональність на основі тої, що існує.

```
class Animal {
 constructor(name) {
   this.speed = 0;
   this.name = name;
 }
 run(speed) {
   this.speed = speed;
   console.log(`${this.name} біжить зі швидкістю ${this.speed}.`);
 }
 stop() {
   this.speed = 0;
   console.log(`${this.name} стоїть.`);
 }
}
let animal = new Animal("Моя тварина");
```

Синтаксис, щоб розширити інший клас

Теорія: ООР

Щоб розширити інший клас, використовують синтаксис: class Child extends Parent

```
class Rabbit extends Animal {
 hide() {
   console.log(`${this.name} ховається!`);
 }
}

let rabbit = new Rabbit("Білий Кролик");
rabbit.run(5); // Білий Кролик біжить зі швидкістю 5.
rabbit.hide(); // Білий Кролик ховається!
```

Об’єкт класу Rabbit має доступ і до методів Rabbit, таких як rabbit.hide(), і до методів Animal.

Ключове слово extends

Теорія: ООР

Внутрішньо, ключове слово extends працює за допомогою механіки прототипу. Він встановлює в Rabbit.prototype.[[Prototype]] значення Animal.prototype. Тому, якщо метод не знайдено в Rabbit.prototype, JavaScript бере його з Animal.prototype.
JavaScript використовує прототипне наслідування для вбудованих об'єктів. Наприклад, Date.prototype.[[Prototype]] – це Object.prototype.
Після слова extends допускається будь-який вираз. Синтаксис класу дозволяє вказати не лише клас, але будь-який вираз після extends. Наприклад, виклик функції, який генерує батьківський клас:

```
function f(phrase) {
 return class {
   sayHi() { console.log(phrase); }
 };
}
class HiUser extends f("Привіт") {}
new HiUser().sayHi(); // Привіт
```

Тут class HiUser успадковує від результату f("Привіт").
Це може бути корисним для просунутих шаблонів програмування, коли ми використовуємо функції для створення класів залежно від багатьох умов і можемо успадкуватися від них.

Перевизначення конструктора

Теорія: ООР

Відповідно до специфікації, якщо клас розширює ще один клас і не має конструктора, то автоматично створюється “порожній” конструктор:

```
class Rabbit extends Animal {
 // генерується для класів-нащадків без власних конструкторів
 constructor(...args) {
   super(...args);
 }
}
```

Він викликає батьківський конструктор та передає йому всі аргументи. Це відбувається, якщо ми не напишемо для нашого класу свій власний конструктор.

Індивідуальний конструктор

Теорія: ООР

Додамо індивідуальний конструктор до Rabbit. Він буде визначати earLength на додачу до name:

Похідний конструктор повинен викликати super, щоб виконати його батьківський (базовий) конструктор, інакше об’єкт для this не буде створено. І ми отримаємо помилку.

```
class Rabbit extends Animal {

 constructor(name, earLength) {
   super(name);
   this.earLength = earLength;
 }

 // ...
}
let rabbit = new Rabbit("Білий Кролик", 10);
alert(rabbit.name); // Білий Кролик
alert(rabbit.earLength); // 10
```

Перевизначення методу

Теорія: ООР

Всі методи, які не вказані в class Rabbit, беруться безпосередньо “як є” від класу Animal.

Але якщо ми вкажемо наш власний метод в Rabbit, наприклад, stop(), то він буде використовуватися замість методу з класу Animal:

```
class Rabbit extends Animal {
 stop() {
   // ...тепер цей метод буде використано для rabbit.stop()
   // замість stop() з класу Animal
 }
}
```

Ключове слово "super"

Теорія: ООР

Зазвичай ми не хочемо повністю замінити батьківський метод, але, радше побудувати метод на його основі, щоб налаштувати або розширити функціональність. Для цього в класах використовують ключове слово "super".

- super.method(...), щоб викликати батьківський метод.
- super(...), щоб викликати батьківський конструктор (лише в нашому конструкторі).

Наприклад, нехай наш кролик автоматично ховається, коли зупиняється:

```
class Rabbit extends Animal {
 hide() {
   alert(`${this.name} ховається!`);
 }
 stop() {
   super.stop(); // викликає батьківський stop
   this.hide(); // а потім ховається
 }
}
let rabbit = new Rabbit("Білий Кролик");
rabbit.run(5); // Білий Кролик біжить зі швидкістю 5.
rabbit.stop(); // Білий Кролик стоїть. Білий Кролик ховається!
```

Тепер Rabbit має метод stop, який в процесі викликає батьківський super.stop().

Стрілкові функції не мають super

Теорія: ООР

Стрілкові функції не мають super.

Якщо super доступний, то він береться із зовнішньої функції. Наприклад:

```
class Rabbit extends Animal {
 stop() {
   setTimeout(() => super.stop(), 1000); // викликає батьківський stop після 1 сек
 }
}
```

super у стрілкової функції такий самий, як у stop(), тому він працює як передбачається. Якщо ми вкажемо тут “звичайну” функцію, то виникне помилка:

```
// Unexpected super
setTimeout(function() { super.stop() }, 1000);
```

Ключове слово static

Теорія: class JavaScript

Ключове слово **static** визначає статичний **метод** або **поле** для класу або **статичний блок ініціалізації**. **Не можна отримати прямий доступ до статичних властивостей** екземплярів класу. Натомість доступ до них здійснюється в самому класі.
Статичні методи часто є **допоміжними функціями**, такими як функції для створення або клонування об’єктів, тоді як **статичні властивості** корисні для кешу, фіксованої конфігурації або будь-яких інших даних, які вам не потрібно копіювати в екземплярах.

```
class Store {
  static get(key) {
      let value = localStorage.getItem(key);
      return value === null ? null : JSON.parse(value);
   }
}
Store.get(‘basket’);
```

Класи і наслідування

Створення власного класу

Створення слайдера на патерні класів (OOP Class)

Практика: ООР

```
<div id="main-gallery" class="gallery-container">
      <div class="main-view-wrapper">
         <button class="nav-btn prev-btn">❮</button>
         <div class="main-view">
            <img class="main-photo" src="/images/product-5.jpg" alt="Головне фото">
         </div>
         <button class="nav-btn next-btn">❯</button>
      </div>
      <div class="thumbnails-list">
         <img class="thumb active" src="/images/product-5.jpg" data-large="/images/product-5.jpg" data-index="0"
            alt="Прев'ю 1">
         <img class="thumb" src="/images/product-6.jpg" data-large="/images/product-6.jpg" data-index="1"
            alt="Прев'ю 2">
         <img class="thumb" src="/images/product-7.jpg" data-large="/images/product-7.jpg" data-index="2"
            alt="Прев'ю 3">
         <img class="thumb" src="/images/product-8.jpg" data-large="/images/product-8.jpg" data-index="3"
            alt="Прев'ю 4">
      </div>
   </div>
```

Стилі слайдера

Практика: ООР

```
.gallery-container {
    max-width: 800px;
    margin: 0 auto;
    text-align: center;
}
.main-view img {
    width: 100%;
    height: auto;
    border-radius: 8px;
    transition: opacity 0.3s ease-in-out;
    /* Плавне зникнення/поява */
}
.thumbnails-list {
    display: flex;
    justify-content: center;
    gap: 10px;
    margin-top: 15px;
}
```

Стилі слайдера

Практика: ООР

```
.thumb {
    width: 120px;
    height: 80px;
    object-fit: cover;
    border-radius: 4px;
    cursor: pointer;
    opacity: 0.6;
    border: 3px solid transparent;
    transition: all 0.2s ease;
}
.thumb:hover {
    opacity: 0.9; /* Стиль для мініатюри, на яку наввели мишкою */
}
/* Стиль для активної мініатюри */
.thumb.active {
    opacity: 1;
    border-color: #007bff; /* Синє підсвічування рамки */
 }
```

Стилі слайдера

Практика: ООР

```
.main-view-wrapper {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 15px;
    position: relative;
}
.main-view {
    flex-grow: 1;
    max-width: 650px;
}
.main-view img {
    width: 100%;
    height: auto;
    border-radius: 8px;
    transition: opacity 0.15s ease-in-out;
}
```

Стилі слайдера

Практика: ООР

```
.nav-btn {
    background: #222;
    color: white;
    border: none;
    font-size: 24px;
    padding: 10px 15px;
    cursor: pointer;
    border-radius: 50%;
    transition: background 0.2s;
}

.nav-btn:hover {
    background: #007bff;
}
```

Головний контейнер та його внутрішні елементи

Практика: ООР

```
class GallerySlider {
        constructor(containerSelector) {
            // 1. Знаходимо головний контейнер та його внутрішні елементи
            this.container = document.querySelector(containerSelector);
            if (!this.container) return;

            this.mainPhoto = this.container.querySelector('.main-photo');
            this.thumbnails = this.container.querySelectorAll('.thumb');
            this.prevBtn = this.container.querySelector('.prev-btn');
            this.nextBtn = this.container.querySelector('.next-btn');
            this.thumbnailsContainer = this.container.querySelector('.thumbnails-list');

            // 2. Внутрішній стан (State)
            this.currentIndex = 0;

            // 3. Запуск ініціалізації подій
            this.initEvents();
        }
```

Метод для оновлення фото та стану активності

Практика: ООР

```
// Метод для оновлення фото та стану активності
updateGallery(index) {
    this.mainPhoto.style.opacity = '0';

    setTimeout(() => {
        const activeThumb = this.thumbnails[index];
        this.mainPhoto.src = activeThumb.getAttribute('data-large');

        // Перемикання класів active
        this.thumbnails.forEach(t => t.classList.remove('active'));
        activeThumb.classList.add('active');

        this.mainPhoto.style.opacity = '1';
    }, 150);
}
```

Навігація вперед/назад

Практика: ООР

```
// Навігація вперед
next() {
    this.currentIndex = (this.currentIndex + 1) % this.thumbnails.length;
    this.updateGallery(this.currentIndex);
}

// Навігація назад
prev() {
    this.currentIndex = (this.currentIndex - 1 + this.thumbnails.length) % this.thumbnails.length;
    this.updateGallery(this.currentIndex);
}
```

Ініціалізація всіх слухачів подій всередині компонента

Практика: ООР

```
// Ініціалізація всіх слухачів подій всередині компонента
initEvents() {
    // Кнопки навігації
    this.nextBtn.addEventListener('click', () => this.next());
    this.prevBtn.addEventListener('click', () => this.prev());

    // Делегація подій для мініатюр
    this.thumbnailsContainer.addEventListener('click', (event) => {
        if (event.target.classList.contains('thumb')) {
            this.currentIndex = Number(event.target.getAttribute('data-index'));
            this.updateGallery(this.currentIndex);
        }
    });
```

Глобальне керування клавіатурою

Практика: ООР

```
        // Глобальне керування клавіатурою (реагує, лише якщо фокус не в інпутах)
        document.addEventListener('keydown', (event) => {
            const activeTag = document.activeElement.tagName;
            if (activeTag === 'INPUT' || activeTag === 'TEXTAREA') return;

            if (event.key === 'ArrowRight') this.next();
            if (event.key === 'ArrowLeft') this.prev();
        });
    }
}
// ========================================================
// ВИКЛИК ТА ЗАПУСК КОМПОНЕНТА
// ========================================================
// Передаємо у конструктор унікальний селектор нашої галереї

const slider = new GallerySlider('#main-gallery');
```

ВПРАВИ

Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Консорціум Всесвітнього павутиння https://www.w3.org/

ECMAScript https://ru.wikipedia.org/wiki/ECMAScript

ECMA http://www.ecma-international.org/

Standard ECMA-262 http://www.ecma-international.org/publications/standards/Ecma-262.htm

JavaScript Web APIs:

- https://www.w3.org/standards/webdesign/script
- https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua